CLIFF Route Runtime

Executable routed orchestration with convergence-aware stopping

Simon Frost

Table of contents

Introduction

The first CLIFF tranche in `FunctorFlow.jl` added route semantics, conscious workspace types, evidence convergence, and artifact records. This vignette shows the next step: turning those semantic pieces into a runnable routed workflow.

The pattern is:

1. create a `CLIFFRuntime`;
2. register a route executor with ordinary Julia code;
3. configure agent capabilities, attention limits, and convergence policy; and
4. run a natural-language query all the way to a `RouteRunResult`.

Setup

```
using Pkg
Pkg.activate(joinpath(@__DIR__, ".."))

using FunctorFlow
```

Build a runtime and register a route executor

`register_route_executor!` accepts a route name and a normal Julia callback. The callback receives the query string and a mutable `CLIFFExecutionContext`. It can publish shared broadcasts, store route-local state, and return one or more `CLIFFRouteUpdate` values.

```
runtime = CLIFFRuntime()

register_route_executor!(runtime, :company_similarity; description="Inline vignette executor")
    set_route_state!(ctx, :normalized_pair, ("Adobe", "Nike"))
    publish!(ctx.board;
        source_agent="retrieval_scout",
        title="Pair recognized",
        summary="The query compares Adobe and Nike across a recent filing window.",
        tags=["company_similarity", "routing"])

    update_1 = CLIFFRouteUpdate(
        snapshot=Dict(
            :top_claims => ["direct-to-consumer", "membership flywheel"],
            :summary => "The first evidence batch isolates a shared channel-expansion theme.",
        ),
        evidence_delta=2,
        processes=[
            UnconsciousProcess(:filing_overlap, "retrieval_scout";
                summary="Two filings already agree on direct-channel emphasis.",
                artifact_refs=["filing-batch-01", "filing-batch-02"],
                salience=0.88, relevance=0.94, novelty=0.41, urgency=0.38, attention_cost=2),
            UnconsciousProcess(:draft_memo, "synthesis_editor";
                summary="A draft memo frame is ready once one more stable batch arrives.",
                artifact_refs=["draft-note-01"],
                salience=0.54, relevance=0.68, novelty=0.26, urgency=0.29, attention_cost=2),
        ],
    )

    publish!(ctx.board;
        source_agent="evidence_judge",
        title="Similarity signal strengthening",
        summary="The first evidence floor has been met; the next stable batch can stop the rou
        tags=["company_similarity", "convergence"],
        audience="synthesis_editor")

    update_2 = CLIFFRouteUpdate(
```

```
        snapshot=Dict(
            :top_claims => ["direct-to-consumer", "membership flywheel"],
            :summary => "A second evidence batch keeps the dominant claims unchanged.",
        ),
        evidence_delta=1,
        processes=[
            UnconsciousProcess(:stable_claims, "evidence_judge";
                summary="The dominant claims are now stable enough to stop gathering.",
                artifact_refs=["stability-report"],
                salience=0.83, relevance=0.92, novelty=0.22, urgency=0.57, attention_cost=2),
            UnconsciousProcess(:final_dashboard, "synthesis_editor";
                summary="The dashboard is ready to be emitted.",
                artifact_refs=["company-similarity-dashboard"],
                salience=0.77, relevance=0.87, novelty=0.35, urgency=0.66, attention_cost=2),
        ],
        artifacts=[
            SynthesisArtifact(:company_similarity_dashboard;
                artifact_kind=:dashboard,
                title="Company Similarity Dashboard",
                summary="A routed comparison between Adobe and Nike over recent filings.",
                content_ref="outputs/company_similarity/company_similarity_dashboard.html",
                tags=["dashboard", "company_similarity"]),
        ],
    )

    [update_1, update_2]
end

println("Registered executors: ", list_route_executors(runtime))
```

```
Registered executors: [:company_similarity]
```

Configure agents, attention, and convergence

The runtime configuration is where the route is tied to:

- an agent pool,
- available execution capabilities,
- a conscious field-of-view, and
- an evidence convergence policy.

```
agents = [
    CLIFFAgentSpec(:retrieval_scout;
        description="Collect routed evidence and surface intermediate findings.",
        instructions="Retrieve and summarize evidence for the active route.",
        required_capabilities=[:llm_inference, :retrieval],
        preferred_capabilities=[:tool_calling, :web_search],
        route_bindings=[:company_similarity]),
    CLIFFAgentSpec(:synthesis_editor;
        description="Write the final routed synthesis artifact.",
        instructions="Synthesize the active route into one coherent answer.",
        required_capabilities=[:llm_inference],
        preferred_capabilities=[:structured_output],
        route_bindings=[:company_similarity]),
    CLIFFAgentSpec(:evidence_judge;
        description="Judge whether routed evidence has converged.",
        instructions="Decide when the route can stop gathering new evidence.",
        required_capabilities=[:llm_inference],
        preferred_capabilities=[:structured_output],
        route_bindings=[:company_similarity]),
]

config = CLIFFRuntimeConfig(
    agents=agents,
    available_capabilities=[:llm_inference, :retrieval, :tool_calling, :structured_output, :we
    consciousness=ConsciousnessFunctor(
        field_of_view=ConsciousFieldOfView(4),
        weights=AttentionScoreWeights(),
    ),
    convergence_policy=EvidenceConvergencePolicy(
        3;
        stability_threshold=0.8,
        required_stable_passes=1,
        max_evidence=6,
    ),
)

println("Route capability gap: ", route_capability_gap(route_cliff_query("How similar is Adobe
println("Selected agents will be filtered at runtime from: ", [agent.name for agent in agents]
```

```
Route capability gap: Symbol[]
Selected agents will be filtered at runtime from: [:retrieval_scout, :synthesis_editor, :evide
```

Execute the routed query

`execute_cliff_query` routes the natural-language prompt, runs the registered executor, updates the conscious workspace, tracks evidence convergence, and returns a `CLIFFRouteTrace`.

```
trace = execute_cliff_query(
    runtime,
    "How similar is Adobe to Nike across recent filings?";
    execution_mode=:deep,
    config=config,
)

summary = summarize_cliff_route_trace(trace)

println("Route: ", summary["route_decision"]["route_name"])
println("Status: ", summary["status"])
println("Selected agents: ", summary["selected_agents"])
println("Counts: ", summary["counts"])
println("Broadcast titles: ", summary["broadcast_titles"])
println("Latest assessment: ", summary["latest_assessment"])
println("Primary artifact: ", summary["primary_artifact"])
```

```
Route: company_similarity
Status: completed
Selected agents: ["retrieval_scout", "synthesis_editor", "evidence_judge"]
Counts: Dict("broadcasts" => 2, "updates" => 2, "assessments" => 2, "artifacts" => 1, "workspa
Broadcast titles: ["Pair recognized", "Similarity signal strengthening"]
Latest assessment: Dict{String, Any}("stability_threshold" => 0.8, "similarity" => 1.0, "evide
to-consumer", "membership flywheel"]), "stop" => true, "required_stable_passes" => 1…)
Primary artifact: Dict{String, Any}("summary" => "A routed comparison between Adobe and Nike o
```

The result is already normalized into a first-class semantic object, so it can be compiled with the rest of the FunctorFlow v1 stack.

```
plan = compile_plan(:CLIFFRuntimeVignettePlan, trace.result; metadata=Dict(:example => "cliff_

compiled_subjects = [(artifact.subject_name, artifact.subject_kind) for artifact in plan.artif
println("Compiled subjects: ", compiled_subjects)
```

```
Compiled subjects: [(:company_similarity__route_run, :cliff_route_run)]
```

Broadcast the selected conscious workspace

The conscious workspace can itself be turned into shared broadcasts.

```
workspace_board = ConsciousBroadcastBoard()
workspace_messages = publish_workspace!(workspace_board, trace.result.workspace; source_agent=

println("Workspace broadcasts: ", [message.title for message in workspace_messages])
println("Workspace payload: ", as_dict(workspace_messages[1])["payload"])
```

```
Workspace broadcasts: ["Selected process: final_dashboard", "Selected process: stable_claims"]
Workspace payload: Dict{String, Any}("score" => 0.6994999999999999, "process_name" => "final_d
similarity-dashboard"])
```

Why this matters

The CLIFF layer is now more than a semantic shell:

- routing is tied to executable Julia callbacks;
- consciousness becomes a concrete bounded-selection mechanism;
- convergence can stop a route automatically; and
- route results slot directly into the semantic compiler.

That makes it possible to prototype whole routed orchestration loops in Julia before attaching real retrieval systems or LLM clients.